



Unraid Watcher

Documentation

Version 1.0.0

© 2026 Ray Munro

Licensed under the GNU General Public License v3.0 or later

Source code: <https://github.com/RayMunro/unraid-watcher>

Contents

1. [Overview](#)
2. [Getting started](#)
3. [User guide](#)
4. [Settings](#)
5. [Alerts and notifications](#)
6. [Controls and safety](#)
7. [Troubleshooting](#)
8. [Data sources](#)
9. [Architecture](#)
10. [Building and releasing](#)
11. [Changelog](#)

Unraid Watcher

A native macOS dashboard and control panel for your Unraid server, with a menu bar status item.

Unraid Watcher shows you what your server is doing (array health, disks, containers, VMs, shares, temperatures, network traffic, logs) and lets you act on it (start and stop containers and VMs, manage the array, run parity checks, create and clone VMs, manage shares) without opening the Unraid web interface.

© 2026 Ray Munro. Licensed under the GNU General Public License v3.0 or later.

Highlights

- **Live overview** of CPU, memory, array state, Docker, VMs, and alerts
- **Disk health:** usage, temperatures, parity history, and SMART details per disk
- **Docker and VM control**, including VM creation, editing, cloning, and console access
- **Share management** with per-disk usage breakdowns
- **Performance and network** views: per-core load, disk I/O, fans, CPU temperatures, and traffic
- **macOS notifications** for hot disks, a hot CPU, SMART problems, and new Unraid alerts
- **Menu bar** summary that is always one click away
- **Secure by default:** the API key and SSH password live in the macOS Keychain

Requirements

Item	Requirement
Mac	macOS 14 (Sonoma) or later
Unraid	7.2 or later (built-in API), or an older version with the Connect plugin
API key	Required. Viewer role to monitor, Admin role to use controls
SSH	Optional, but needed for temperatures, network traffic, SMART, logs, and the VM and share tools

Quick start

1. Build the app and open it (see [Building and releasing](#)), or install it from the DMG.
2. Open **Settings** (Cmd+,).
3. Enter your server address, for example `http://192.168.1.10`, and your API key.
4. Optionally turn on SSH and enter your SSH user and password.
5. Click **Save & Connect**.

The full walkthrough is in [Getting started](#).

Documentation

Guide	What it covers
Getting started	Creating an API key, enabling SSH, first connection
User guide	Every tab and what each control does
Settings	All preferences, defaults, and where data is stored
Alerts and notifications	What triggers a notification and how to tune it
Controls and safety	Destructive actions, confirmations, and permissions
Troubleshooting	Common errors and how to fix them
Data sources	Every API query, mutation, and SSH command the app uses
Architecture	How the code is organized
Building and releasing	Building the app, icon, and DMG installer
Changelog	Version history

All of these guides are also combined into one PDF: [Unraid-Watcher-Documentation.pdf](#). The installer includes a copy.

Privacy and security

- Unraid Watcher talks only to the server address you enter. It has no analytics, no telemetry, and no third-party services.
- Your API key and SSH password are stored in the macOS Keychain, never in plain text on disk.
- Destructive actions (stopping the array, rebooting, deleting a share, force-stopping a VM) always ask for confirmation first.

License

Unraid Watcher is free software: you can redistribute it and modify it under the terms of the GNU General Public License as published by the Free Software Foundation, either version 3 of the License, or (at your option) any later version.

It is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the [LICENSE](#) file for the full text.

© 2026 Ray Munro. Licensed under the GNU General Public License v3.0 or later.

Disclaimer

This is an independent project. It is not affiliated with, endorsed by, or sponsored by Lime Technology, Inc. Unraid is a trademark of Lime Technology, Inc. This software can change settings, stop services, and delete share and VM definitions on your server. Use it at your own risk and keep backups, as the license above makes clear.

Getting started

This guide takes you from a fresh install to a connected dashboard.

1. Prepare your Unraid server

Create an API key

Unraid Watcher reads data through Unraid's GraphQL API, which needs an API key.

1. Open the Unraid web interface.
2. Go to **Settings, Management Access, API Keys**.
3. Create a new key.
4. Choose a role:
 - **Viewer** is enough to monitor everything on the dashboard.
 - **Admin** is needed for actions that change things through the API: starting and stopping containers, the array, parity checks, VM power actions, and archiving notifications.
5. Copy the key. You will paste it into Unraid Watcher.

Unraid 7.2 and later include the API. On older versions, install the Unraid Connect plugin to get it.

Enable SSH (optional but recommended)

Some features read data or run commands directly on the server over SSH:

- CPU temperatures, fan speeds, network traffic, load, memory detail, disk I/O, and top processes
- SMART health details and self-tests
- The system log viewer and container logs
- Spinning disks up or down, the mover, reboot and shutdown
- VM details, creation, editing, cloning, media, and console access
- Share settings and share management
- The command console

To enable it, open **Settings, Management Access** in Unraid and turn on **Use SSH**. Make sure you know the root password, or set up key-based login.

Find your server address

Use the same address you type in a browser to open the Unraid web interface, for example `http://192.168.1.10` or `http://tower.local`. If you use HTTPS with a self-signed certificate, see [Settings](#).

2. Connect Unraid Watcher

1. Open Unraid Watcher.
2. Open **Settings** with Cmd+, (or the app menu).
3. Fill in:
 - **Server URL**: your server address
 - **API key**: the key you created
4. To use SSH features, turn on **Read temperatures & network over SSH** and fill in:
 - **SSH user**: usually `root`
 - **SSH password**: your server password, or leave it empty to use your Mac's SSH keys
5. Click **Save & Connect**.

macOS may ask for your login keychain password the first time the app reads or writes the saved API key or SSH password. Enter it and choose **Always Allow**. This is a normal macOS prompt, and the app never sees your keychain password.

3. Check that it works

- The **Overview** tab should show your hostname, CPU, memory, and array state within a few seconds.
- A small timestamp ("Updated ...") appears in the toolbar after each successful refresh.
- Any problems appear as orange messages at the top of the page. Click **Copy all errors** to copy them. See [Troubleshooting](#) for what they mean.

SSH with keys instead of a password

If you prefer keys, leave the SSH password empty and run this once in Terminal:

```
ssh-copy-id root@your-server-address
```

Unraid Watcher uses your Mac's SSH keys when no password is set. With a password set, it still tries keys first and then falls back to the password.

Next steps

- Read the [User guide](#) for a tour of each tab.
- Tune thresholds in [Alerts and notifications](#).
- Review [Controls and safety](#) before using the power and array controls.

© 2026 Ray Munro. Licensed under the GNU General Public License v3.0 or later.

User guide

The window has a sidebar on the left and a detail area on the right. A footer at the bottom of the sidebar shows the app name and copyright. Click it to open the About window.

The **refresh button** in the toolbar updates everything immediately. The app also refreshes on its own every few seconds (see [Settings](#)).

Features marked **SSH** need SSH turned on in Settings. Features marked **Admin** need an API key with Admin rights.

Overview

A summary of the whole server:

- **Temperature alerts:** appears only when a disk is at or above your warning temperature.
- **System:** hostname, Unraid version, CPU model, core and thread counts, and boot time.
- **CPU and Memory:** current load with a graph of the last 60 refreshes. With SSH on, the CPU card also shows the CPU temperature.
- **Array:** state, used and free space, and the hottest disk.
- **Docker:** running containers out of the total.
- **VMs:** running VMs out of the total.
- **Alerts:** unread notification counts split into alert, warning, and info.

System & Power

- **Hardware:** system and motherboard make and model, CPU socket and speed, architecture.
- **Software:** Unraid, kernel, and API versions.
- **License & server:** license type and expiry, flash drive vendor and product, time zone, array disk count, share count, and safe mode.
- **Services:** whether each Unraid service is online.
- **UPS:** status, battery charge, estimated runtime, load, and voltages. If no UPS is attached, the card says "No UPS detected".

Network & Temps (SSH)

- **CPU temperature:** a large reading, a history graph, and each core or package sensor. The reading turns orange above 70 C and red above 80 C.
- **Other sensors:** NVMe and any other temperature sensors the server reports.
- **Network traffic:** combined download and upload rates, a graph, and a per-interface breakdown. Virtual interfaces such as `lo`, `veth*`, `docker*`, and `virbr*` are hidden. Rates need two samples, so they appear after the second refresh.

Performance (SSH)

- **Load and uptime:** 1, 5, and 15 minute load averages and uptime.
- **CPU cores:** a usage bar for every core.
- **Memory:** usage with a breakdown of in-use, cache and buffers, available, shared, and dirty memory, plus swap if configured.
- **Disk I/O:** read and write speed for every physical disk, labeled with the Unraid disk name.
- **Fans:** fan speeds reported by the motherboard.
- **Top processes:** the busiest processes by CPU.

SMART Health (SSH)

One card per disk showing:

- A health badge: **Healthy**, **Warning**, **Failing**, **Standby**, or **Unknown**
- Temperature, model, serial, firmware, capacity, drive type, power-on time, power cycles, last self-test result, and error log entries
- Any concerns found (see below)
- A full table of SMART attributes (NVMe drives show wear, spare capacity, media errors, and unsafe shutdowns instead)

Buttons per disk: **Short self-test**, **Extended self-test** (asks first, can take hours), **Spin down**, and **Spin up**.

How health is decided:

- **Failing:** the drive reports an overall SMART failure, or any attribute has failed.
- **Warning:** reallocated, pending, offline-uncorrectable, or reallocation-event sector counts above zero; NVMe media errors; critical warning flags; SSD wear of 90 percent or more; or spare capacity below its threshold.
- **Healthy:** the drive passes with no concerns.

Disks that are spun down are never woken for a SMART check. Their last known data is kept and they show a "Spun down" badge. The tab checks automatically every 30 minutes, or when you click **Check now**.

Controls (SSH and Admin, depending on the action)

- **Array:** shows the state with **Start array** and **Stop array**. During a parity check it shows progress, errors, and **Pause**, **Resume**, and **Cancel**. When idle you can start a read-only check or a correcting check.
- **Disks and mover (SSH):** spin all disks up or down, and start or stop the mover.

- **Power (SSH): Reboot and Shut down.** Both ask for confirmation and use Unraid's clean powerdown, which stops the array first. A shut-down server cannot be turned back on from the app.
- **Console (SSH):** run any command on the server and see its output. Recent commands are kept as shortcuts.

Array & Disks

- **Array card:** state and total capacity, then each parity, data, and cache disk with its status, temperature, usage bar, and details (filesystem, spinning or spun down, HDD or SSD, read and write counts, and error count).
- **Mounted filesystems (SSH):** the flash drive, Docker image, log, and `/mnt` mounts with usage.
- **Parity check history:** the last 10 checks with date, duration, speed, and errors.

Disk temperatures use the warning and critical thresholds from Settings.

Docker

Every container with its image, state, ports, auto-start badge, and created date.

- **Play and stop buttons:** start a stopped container, or stop a running one (asks first). Admin.
- **Menu (the three dots):**
 - **Restart:** uses SSH if enabled, otherwise stops and starts through the API.
 - **Pause and Resume.**
 - **View logs (SSH):** the last 300 lines, with a refresh button.

VMs

Every VM with its state. The three-dot menu on each VM offers:

- **Details (SSH):** CPU and memory summary, disks, network interfaces, VNC display, the last 60 lines of the VM log, and a **Start with array** switch.
- **Edit (SSH):** three tabs.
 - **Basics:** change vCPUs and memory.
 - **Media:** see each CD or DVD drive, eject a disc, or insert an ISO from `/mnt/user/isos`.
 - **XML:** edit the full libvirt definition. libvirt validates every save, and a bad edit is rejected.
 - **Remove VM:** deletes the definition only. Disk images are kept, and the VM must be shut off.
- **Clone (SSH):** copy a shut-off VM and its disks under a new name. See below.
- **Eject install media:** one click to eject every disc.
- **Start, Shut down, Pause, Resume, Reboot, Force stop:** power actions. Reboot and force stop ask first. Admin.
- **Open console (Screen Sharing) and Open console in browser:** for running VMs.

Creating a VM

Click **New VM** above the list. Choose:

- Name, operating system (Linux or Windows 10 / Server), and firmware (UEFI or legacy BIOS)
- vCPUs and memory
- A new disk (folder and size) or an existing disk image
- Disk bus: VirtIO (fastest, but Windows needs drivers) or SATA (works everywhere)
- An install ISO, and for Windows a VirtIO drivers ISO
- Network bridge and adapter type
- Whether to start with the array and whether to start the VM now

Choosing Windows switches to a SATA disk and an Intel network adapter so the installer works without extra drivers. The VM is created as a standard KVM VM with VNC graphics. GPU and USB passthrough and a TPM (needed for Windows 11) can be added afterwards in the XML editor.

If defining the VM fails, any disk image the app just created is removed. An untouched empty image left by an earlier failed attempt is replaced automatically.

Cloning a VM

The source VM must be shut off. The clone gets:

- A new name and a new unique ID
- Copies of every file-backed disk, made sparse so unused space is not copied
- A new network address (MAC addresses are removed so libvirt generates new ones)
- Its own copy of the UEFI variable store

VMs that use a physical disk device cannot be cloned automatically. Passthrough devices are copied as-is, and two VMs cannot use the same device at once. Large disks can take a long time, so keep the window open while it runs.

Opening the console

- **Screen Sharing** opens the VM in the built-in macOS Screen Sharing app using the VM's VNC port. This is the most reliable option.
- **Browser** opens Unraid's built-in web VNC page for the VM.

The VM must be running.

Shares

A list of shares with used and free space. Click a share to expand it.

- **Settings (SSH):** cache use, allocation method, split level, minimum free space, included and excluded disks, and SMB export and security.

- **Analyze disk usage:** a per-disk breakdown of where the share's data lives. Runs on demand because it can be slow on large shares.
- **New share:** a form for name, comment, cache use, allocation, split level, minimum free space, disk include and exclude lists, and SMB export and security.
- **Edit:** the same form for an existing share. Settings that the form does not cover are preserved.
- **Delete:** only for empty shares. The app checks every disk first and refuses if any files remain.

Share changes are submitted to Unraid's own web interface endpoint from the server itself, the same way the web interface saves them. If the server does not accept a change, a banner says so.

Notifications

Unread Unraid notifications with their importance, message, and time. Archive one with the archive button, or use **Archive all**. Admin.

Logs (SSH)

The last 60 lines of the system log, with errors in red and warnings in orange. The text can be selected and copied.

Menu bar

The menu bar icon shows a quick summary: hostname, array state, CPU and memory, Docker count, and unread notifications. From it you can open the dashboard, open Settings, open the About window, or quit. The icon changes when the server cannot be reached.

Banners

Results of actions appear in a banner at the top of the window. Green means success, red means a failure with the server's message. Use the copy button on the banner to copy the text.

About window

Choose **About Unraid Watcher** from the app menu, the menu bar panel, or the sidebar footer. It shows the version and the copyright notice.

© 2026 Ray Munro. Licensed under the GNU General Public License v3.0 or later.

Settings

Open Settings with Cmd+, or from the menu bar panel. Changes to the connection take effect when you click **Save & Connect**.

Connection

Setting	Default	Notes
Server URL	empty	The address you use for the Unraid web interface, for example <code>http://192.168.1.10</code> . If you leave off <code>http://</code> , the app adds it.
API key	empty	Stored in the macOS Keychain. Viewer role to monitor, Admin role to use actions.
Allow self-signed certificate	on	Lets the app connect to an HTTPS server whose certificate is not trusted by macOS. This only applies to the server address you entered.
Refresh every N s	10	How often data is refreshed. Range: 2 to 300 seconds. SSH features add a little work per refresh, so very short intervals may feel heavy on a small server.

The app allows plain HTTP, so a normal home setup works without certificates. On HTTP, the API key travels unencrypted across your network. Use a trusted local network, or set up HTTPS on your server and enter an `https://` address.

SSH

Setting	Default	Notes
Read temperatures & network over SSH	off	Master switch for every SSH-based feature.
SSH user	root	The account used to log in.
SSH password	empty	Optional. Stored in the Keychain. Leave it empty to use your Mac's SSH keys.

How SSH login works:

- With no password set, the app runs `ssh` in batch mode and relies on your Mac's keys.
- With a password set, the app still offers your keys first, then falls back to the password. The password is passed to `ssh` through a small temporary helper script that reads it from the environment. It is never written to disk and never placed on a command line.
- The first connection to a server accepts and remembers its host key automatically.

Alerts

Setting	Default	Range	Notes
Notify about alerts	on		Turns all macOS notifications on or off.
Disk warning at	45 C	30 to 70	A disk at or above this temperature is shown as hot and triggers a warning notification.
Disk critical at	55 C	35 to 80	A disk at or above this triggers a critical notification.
CPU warning at	80 C	50 to 100	Needs SSH, because CPU temperature comes from the server's sensors.
Send test notification			Sends a sample notification and asks macOS for permission if needed.

See [Alerts and notifications](#) for exactly when notifications fire.

Where data is stored

Data	Location
API key	macOS Keychain, item name <code>apiKey</code>
SSH password	macOS Keychain, item name <code>sshPassword</code>
Server URL, SSH user, thresholds, refresh interval, toggles	App preferences (<code>com.raymondmunro.unraidwatcher</code>)
SSH password helper	A temporary script in the system temp folder, containing no secrets

Nothing is sent anywhere except to your own server.

Migrating from earlier names

The app was developed under earlier names. On first launch, it copies any preferences it finds from those earlier versions. Keychain items keep their names, so saved secrets carry over. macOS may ask once for your login keychain password to allow the renamed app to read them.

Resetting everything

To clear all preferences:

```
defaults delete com.raymondmunro.unraidwatcher
```

To remove saved secrets, open **Keychain Access**, search for `apiKey` and `sshPassword` , and delete those items.

© 2026 Ray Munro. Licensed under the GNU General Public License v3.0 or later.

Alerts and notifications

Unraid Watcher sends standard macOS notifications. Turn them on or off, and set temperature limits, in [Settings](#).

What triggers a notification

Disk temperature

For every disk (parity, data, and cache) the app compares the temperature with your limits:

- At or above the **warning** limit (default 45 C): a "running hot" notification.
- At or above the **critical** limit (default 55 C): a "critical" notification.

To avoid repeated alerts, a disk must cool to **3 degrees below** the limit before it can alert at that level again. A disk hovering around the limit therefore notifies once, not every refresh.

The Overview tab also shows a Temperature alerts card for any disk at or above the warning limit, and disk temperatures are colored orange or red on the Array & Disks tab.

CPU temperature (needs SSH)

When the hottest CPU sensor reaches the CPU limit (default 80 C), you get one notification. It re-arms after the temperature drops 3 degrees below the limit.

SMART health (needs SSH)

When a disk's SMART health changes to **Warning** or **Failing**, you get a notification naming the disk and the first concern found. The same state does not notify twice. SMART is checked every 30 minutes, and whenever you click **Check now** on the SMART Health tab.

Unraid notifications

New unread notifications with **alert** or **warning** importance are forwarded. Informational notifications are not. On the first refresh after launch, existing notifications are recorded silently so you are not flooded with old messages.

Requirements and limits

- macOS asks for notification permission the first time the app launches. If you declined, enable it in **System Settings, Notifications, Unraid Watcher**.
- Notifications are sent only while the app is running. The app normally keeps running in the menu bar when its window is closed. Choose **Quit** from the menu bar panel to stop it.

- If the server is unreachable, the app cannot see problems on it. A disconnected server shows as a crossed-out icon in the menu bar.

Testing

Open Settings and click **Send test notification**. If nothing appears, check System Settings, Notifications, and make sure Focus modes are not hiding them.

Tuning tips

- Spinning hard drives are usually comfortable up to the mid 40s C. Many people use 45 C for warning and 55 C for critical, which are the defaults.
- SSDs and NVMe drives run hotter by design. If you get frequent warnings for a cache drive, raise the warning limit slightly or improve airflow.
- Your drives' own datasheets give their rated maximum temperature. Keep the critical limit safely below it.

© 2026 Ray Munro. Licensed under the GNU General Public License v3.0 or later.

Controls and safety

Unraid Watcher can change things on your server, not just read them. This guide explains what each action does, what it needs, and what protects you from mistakes.

Permissions

Method	Needed for
Viewer API key	All read-only dashboards
Admin API key	Container start, stop, pause, and resume; VM power actions; array start and stop; parity checks; archiving notifications
SSH	Disk spin up and down, mover, reboot and shutdown, container restart and logs, SMART tests, VM details, creation, editing, cloning, media, and console, share settings and management, the command console

If you only want to monitor, use a Viewer key and leave SSH off. Nothing in the app can then change your server.

Actions that ask for confirmation

Action	What the confirmation warns about
Stop a container	The container will stop
Stop the array	Shares, containers, and VMs become unavailable
Start a correcting parity check	Parity is rewritten to match the data disks
Spin down all disks	Disks in use spin up again on next access
Reboot the server	All services go down briefly
Shut down the server	The server powers off and cannot be turned on from the app
Reboot a VM	The VM restarts
Force stop a VM	Equivalent to pulling the plug
Extended SMART self-test	Can take hours and slows the disk
Delete a share	Removes the share definition
Remove a VM	Removes the definition only, disks are kept

Actions that are easy to undo, such as starting a container, pausing a VM, or a short SMART test, run immediately.

Built-in safeguards

- **Shares:** delete works only on empty shares. The app checks every disk and refuses if any file remains.
- **VMs:** removing a VM never deletes its disk images. Cloning requires the source VM to be shut off. Editing a VM definition is validated by libvirt, and a bad definition is rejected without changing the VM.
- **Failed VM creation:** if defining a VM fails, any new disk image the app created is removed. An existing disk image with data in it is never overwritten.
- **Spun-down disks:** SMART checks use a standby-aware mode so sleeping disks are not woken.
- **Input checks:** share names, VM names, disk device names, and file paths are validated, and everything passed to a shell command is quoted. Names with unusual characters are refused rather than risked.
- **Power commands** use Unraid's own powerdown, which stops the array cleanly before turning off or rebooting.

The command console

The Console on the Controls tab runs exactly what you type, as the SSH user, with no filtering and no confirmation. It is as powerful as a terminal on the server. Treat it that way.

Reversibility

Action	Easy to undo?
Start, stop, pause, or restart a container or VM	Yes
Spin disks up or down	Yes
Stop the array	Yes, start it again
Parity check (read-only)	Yes, cancel it
Parity check (correcting)	No, parity is rewritten
Force stop a VM	Risky, the guest may be left inconsistent
Delete an empty share	Re-create it
Remove a VM definition	Re-create it against the same disk
Shut down the server	Needs physical or out-of-band power on
Command console	Depends entirely on the command

Good habits

- Try new features on a test VM or an empty test share first.

- Use shut down rather than force stop for VMs whenever possible.
- Keep an up-to-date backup of your flash drive. VM and share definitions live there.
- After editing a VM's XML, check the VM in the Unraid web interface the first time.

© 2026 Ray Munro. Licensed under the GNU General Public License v3.0 or later.

Troubleshooting

Every page shows problems as orange messages at the top. Problems with optional extras (license, services, UPS, parity history, and per-disk or per-container details) are grouped into one grey, collapsed line so they do not clutter the page. Click **Copy all errors** to copy every message at once.

Connection problems

"A server with the specified hostname could not be found"

The **Server URL** in Settings is wrong, or your Mac cannot resolve the name. Use the same address you type in a browser to reach Unraid, for example `http://192.168.1.10`. Try the IP address instead of a name if `tower.local` does not resolve.

"Unauthorized (HTTP 401)" or "(HTTP 403)"

The API key is wrong, expired, or lacks permission. Create a new key in Unraid under **Settings, Management Access, API Keys**, paste it into Settings, and click **Save & Connect**.

Timeouts or "could not connect"

- Check that the server is on and reachable: open its web interface from your Mac.
- Confirm the address, and the port if you changed it (for example `http://192.168.1.10:8080`).
- If you use HTTPS with a self-signed certificate, make sure **Allow self-signed certificate** is on.

The window just says "Connect to your server"

No URL or API key is saved yet. Open Settings and fill in both.

Unexpected messages that are not real problems

Message	Meaning
"No UPS data returned from apcaccess"	No UPS is attached. The app shows "No UPS detected" instead of an error.
Optional details unavailable	Your Unraid version does not offer one of the extra data fields. Everything else keeps working.

API data errors

Each panel fetches its data separately, so one failing field affects only its own panel.

"Cannot return null for non-nullable field ..."

Your server returned nothing for a field that the API says must always exist. This is a server-side quirk with that field. The app has been adjusted to avoid the fields known to do this (for example the flash drive GUID). If you see another one, copy the message and report it.

A panel says a field "doesn't exist" or the query is invalid

The Unraid API changes between versions, and a few panels use fields that older or newer versions may not offer. Update Unraid to 7.2 or later if you can. Copy the message so the query can be adjusted.

Actions fail with a red banner

Actions that go through the API (container start and stop, VM power, array, parity check, archive notification) need an **Admin** API key. A Viewer key can read but not change anything. The banner shows the server's own message, and the copy button on the banner copies it.

SSH problems

The SSH error message is shown exactly as `ssh` reported it.

Message	Fix
Could not resolve hostname	The Server URL is wrong. SSH uses the same host.
Permission denied	Wrong user or password. Check the SSH user and password in Settings.
Connection refused	SSH is off on the server. Turn on Use SSH in Unraid, Settings, Management Access.
Connection timed out	The server is unreachable or a firewall blocks port 22.
Host key verification failed	The server was rebuilt and its key changed. Remove the old entry from <code>~/.ssh/known_hosts</code> on your Mac.

To test SSH outside the app:

```
ssh root@your-server-address
```

If that works in Terminal but not in the app, check that the user and password in Settings match.

Keychain prompts

macOS asks for your login keychain password when the app reads a saved secret for the first time after it was rebuilt or renamed. Enter it and choose **Always Allow**. If you build the app yourself, signing with a stable certificate (the build script does this automatically when an Apple Development certificate is

installed) stops the prompt from coming back after each rebuild. Ad-hoc signed builds look like a different app every time.

Notifications

If nothing appears:

1. Click **Send test notification** in Settings.
2. Check **System Settings, Notifications, Unraid Watcher** and allow alerts.
3. Make sure a Focus mode is not suppressing them.
4. Confirm **Notify about alerts** is on.

Data that looks empty

Symptom	Reason
Network rates empty	Rates need two samples. Wait for one more refresh.
CPU core bars and disk I/O empty	Same, they are computed from the difference between two samples.
SMART shows "Standby"	The disk is spun down and was deliberately not woken. Its last data is kept when available.
Temperatures missing	SSH is off, or the server exposes no sensors the app recognizes.
Share settings missing	SSH is off, or the share uses defaults and has no settings file.

VM problems

Creating a VM failed

The banner shows libvirt's own message. Common causes:

- **"A VM named ... already exists":** pick another name.
- **"A disk image already exists at ...":** a disk with data is already at that path. Choose another folder or name, or use the existing-image option.
- **A bridge or network error:** pick one of the bridges listed in the form. They come from your server.
- **Missing UEFI firmware files:** choose legacy BIOS, or check that your Unraid version includes OVMF.
- **Permission or path errors:** check the disk folder exists on a share that can hold VM disks.

Older builds of the app could fail with "per-device boot elements cannot be used together with os/boot elements". That was fixed. Update to the latest version.

If creation fails after the disk image was made, the app deletes the new image. An empty image left by an earlier failed attempt is replaced automatically.

Clone failed

- The source VM must be shut off.
- VMs that use a physical disk device cannot be cloned automatically.
- Check there is enough free space for the copied disks.

The browser console button opens a blank or broken page

Use **Open console (Screen Sharing)** instead. It connects straight to the VM's VNC port and does not depend on web interface paths.

Edited XML was rejected

libvirt refuses invalid definitions and leaves the VM unchanged. The banner shows what it disliked. Common causes are a CPU topology that no longer matches the vCPU count, or references to CPUs that were removed.

Shares

"The server didn't accept the change"

Share changes are posted to Unraid's own web interface endpoint from the server. If that fails, check the change in the Unraid web interface to confirm what state the share is in, then try again. Share names can contain letters, numbers, spaces, underscores, hyphens, and periods.

Delete is refused

The share still contains files on at least one disk. Empty it first. This protects your data.

Building problems

- **Codesign cannot find an identity:** the script falls back to ad-hoc signing automatically. Install an Apple Development certificate (free with an Apple ID in Xcode) for stable signing.
- **The DMG window is not styled:** macOS asks to let your terminal control Finder the first time. Allow it, or accept the plain DMG, which still installs normally.
- **Gatekeeper blocks the app on another Mac:** right-click the app and choose Open the first time. Removing the warning for everyone needs a Developer ID certificate and notarization.

Still stuck?

Click **Copy all errors** and, if relevant, copy the banner text of the failing action. Those two pieces of text usually identify the problem.

© 2026 Ray Munro. Licensed under the GNU General Public License v3.0 or later.

Data sources

This page lists everything Unraid Watcher asks of your server, so you can see exactly what it does and audit it. There are two channels: the Unraid GraphQL API and SSH.

GraphQL API

All requests go to `<server>/graphql` over HTTP or HTTPS with the API key in the `x-api-key` header.

Queries (read only)

Panel	Data requested
Overview	<code>info</code> (OS hostname, distribution, release, uptime; CPU brand, cores, threads) and <code>metrics</code> (CPU percent, memory total, used, percent)
Array	<code>array</code> (state, capacity, parities, disks, caches with name, device, size, temperature, status, filesystem usage)
Docker	<code>docker.containers</code> (id, names, state, status, image)
Container details	<code>docker.containers</code> (id, auto-start, created, ports)
VMs	<code>vms.domains</code> (id, name, state)
Shares	<code>shares</code> (name, free, used, size, comment)
Notifications	<code>notifications</code> (unread counts and the 20 newest unread items)
System details	<code>info</code> (baseboard, system, OS kernel and architecture, CPU manufacturer, speed, socket, versions)
Disk details	<code>array</code> disks (filesystem type, errors, spinning state, reads, writes, rotational)
UPS	<code>upsDevices</code> (name, model, status, battery, power)
Parity history	<code>parityHistory</code> (date, duration, speed, status, errors)
Parity status	<code>array.parityCheckStatus</code> (running, paused, progress, errors, status)
License	<code>registration</code> (type, state, expiration)
Services	<code>services</code> (name, online, version)
Flash	<code>flash</code> (vendor, product)
Server	<code>vars</code> (name, version, registration, time zone, disk and share counts, array filesystem state, safe mode)

Each query runs independently and concurrently on every refresh, so a failure in one does not affect the others.

Mutations (changes, need Admin)

Action	Mutation
Start, stop, pause, resume a container	<code>docker { start, stop, pause, unpause }</code>
VM power	<code>vm { start, stop, pause, resume, forceStop, reboot }</code>
Start or stop the array	<code>array { setState(desiredState: START or STOP) }</code>
Parity check	<code>parityCheck { start(correct), pause, resume, cancel }</code>
Archive a notification	<code>archiveNotification(id)</code>

SSH

Commands run as the SSH user through `sh -s`, with the script sent on standard input.

Polled on every refresh (when SSH is on)

A single connection reads, in one batch:

- `/proc/net/dev` for network counters
- `/sys/class/hwmon` for temperatures and fan speeds
- `/proc/loadavg` and `/proc/uptime`
- `/proc/meminfo` for memory detail
- `/proc/stat` for per-core CPU
- `df` for mounted filesystems
- `/proc/diskstats` for disk I/O
- `ps` for the top processes
- the last 60 lines of `/var/log/syslog`

On demand

Feature	Commands
SMART check	<code>smartctl -j -a -n standby /dev/<device></code> for each disk (the standby option leaves sleeping disks alone)
SMART self-test	<code>smartctl -t short</code> or <code>smartctl -t long</code>
Spin down	<code>hdparm -y /dev/<device></code>
Spin up	a single-sector direct read with <code>dd</code>
Mover	<code>mover start</code> , <code>mover stop</code>
Power	<code>powerdown</code> or <code>powerdown -r</code> , started in the background
Container logs and restart	<code>docker logs --tail 300</code> , <code>docker restart</code>

Feature	Commands
Console	whatever you type
Share settings	reading <code>/boot/config/shares/*.cfg</code>
Share disk usage	<code>du -sk /mnt/*/<share></code>
Share create, edit, delete	a request to Unraid's own web interface endpoint on the server (<code>/update.htm</code> , over its local socket when available), authenticated with the server's own token
Share delete safety check	<code>find</code> over the share's folders on each disk
VM details	<code>virsh dominfo</code> , <code>domblklist</code> , <code>domiflist</code> , <code>vncdisplay</code> , and the libvirt VM log
VM autostart	<code>virsh autostart</code>
VM create	<code>virsh dominfo</code> (name check), <code>qemu-img create</code> , <code>virsh define</code> , optional <code>virsh autostart</code> and <code>virsh start</code>
VM edit	<code>virsh dumpxml --inactive</code> , <code>virsh define</code>
VM remove	<code>virsh domstate</code> , <code>virsh undefine --nvram</code>
VM media	<code>virsh domblklist --details</code> , <code>virsh change-media</code>
VM console	<code>virsh dumpxml</code> to read the VNC and websocket ports
VM clone	<code>virsh dominfo</code> , <code>virsh domstate</code> , <code>cp --sparse=always</code> , <code>virsh define</code>
ISO and bridge lists	<code>ls /mnt/user/isos</code> , and the bridge devices under <code>/sys/class/net</code>

Values that come from you or from the server (names, paths, device names) are validated and shell-quoted before they are used in a command.

What stays on your Mac

- Fetched data is held in memory only. Nothing is written to disk except your settings and the Keychain items.
- The app contacts no server other than the one you enter.

© 2026 Ray Munro. Licensed under the GNU General Public License v3.0 or later.

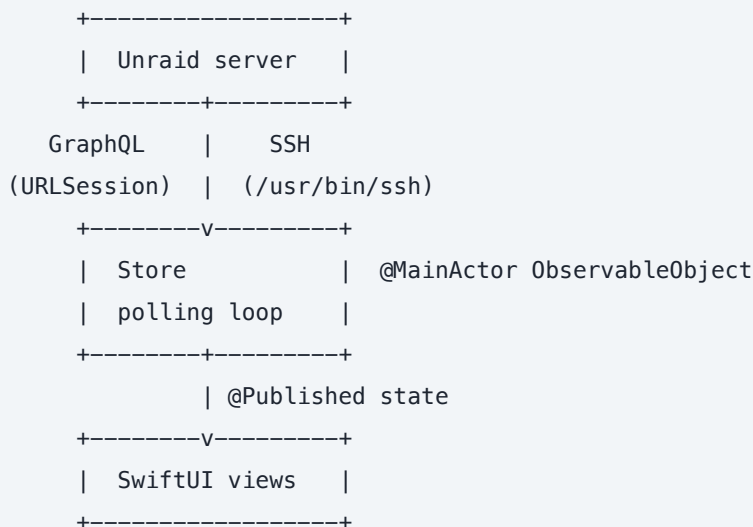
Architecture

Unraid Watcher is a single-target Swift Package that builds a SwiftUI macOS app. It has no third-party dependencies. It uses only Apple frameworks (SwiftUI, Charts, AppKit, Security, UserNotifications, Foundation).

Project layout

Package.swift	Swift package (macOS 14, one executable target)
build_app.sh	Builds and signs "Unraid Watcher.app"
make_dmg.sh	Builds the DMG installer
Resources/AppIcon.icns	App icon
tools/make_icon.swift	Draws the icon and writes the .icns
tools/make_dmg_bg.swift	Draws the DMG window background
tools/make_docs_pdf.py	Builds the documentation PDF from the markdown files
docs/assets/icon.png	App icon used in the docs and the PDF cover
docs/Unraid-Watcher-Documentation.pdf	The combined documentation
Sources/UnraidWatcher/	
App.swift	App entry, scenes, About panel, settings migration
Client.swift	GraphQL client, API models, Keychain helper
Store.swift	Central state, polling loop, alerts
Remote.swift	SSH execution and parsing of server output
Actions.swift	Actions (API and SSH), SMART models and parser
ShareVM.swift	Share management and VM detail views
VMEditor.swift	VM creation, XML generation, edit sheet
VMEExtras.swift	VM clone planning, media, console
Views.swift	Sidebar, all tab views, menu bar, settings

Data flow



The Store

`Store` is the single source of truth. A `Task` loop calls `refresh()` every N seconds.

- **API data:** each GraphQL query runs as its own concurrent task (`async let`). Each returns a `Result` , so a failing query records an entry in the `errors` dictionary without affecting the others. Views show errors relevant to the current tab; errors for optional extras are grouped into a collapsed notice.
- **SSH data:** if SSH is enabled, one `ssh` call fetches many sections at once. The output is parsed into a `RemoteSample` . Rates (network, disk I/O, per-core CPU) are computed from the difference between the current and previous samples and the elapsed time.
- **History:** short arrays (the last 60 samples) feed the sparkline charts.
- **Alerts:** after each refresh, `checkAlerts()` compares temperatures and notifications against thresholds and sends macOS notifications. A small state machine with hysteresis prevents repeats.
- **SMART:** loaded separately every 30 minutes when SSH is on, or on demand.

Talking to the server

- `UnraidClient` (in `Client.swift`) posts GraphQL documents with an `x-api-key` header. It tolerates self-signed certificates only when the setting is on. Responses decode through `Envelope<T>` , and numeric fields that Unraid sometimes returns as strings decode through the `Flex` and `LStr` types.
- `Remote.exec` (in `Remote.swift`) runs `/usr/bin/ssh` as a subprocess and sends the script on standard input to `sh -s` . Standard error is drained on a separate thread to avoid pipe deadlocks. In key mode it uses batch mode. In password mode it uses a temporary `SSH_ASKPASS` helper that reads the password from an environment variable. `ServerAliveInterval` keeps long operations such as VM cloning connected.

- Anything interpolated into a shell script goes through `shq()` (single-quote escaping), and names, paths, and device names are validated first (`validShareName` , `validPath` , `validDev`).

Parsers and generators

These are written as pure functions so they are easy to test in isolation:

Function	Purpose
<code>Remote.parse</code>	Turns the batched SSH output into a <code>RemoteSample</code>
<code>Smart.parse</code>	Turns <code>smartctl -j</code> JSON into a <code>SmartInfo</code> with a health verdict
<code>Store.parseCfg</code>	Parses Unraid share <code>.cfg</code> files
<code>domainXML</code>	Generates a libvirt domain definition for a new VM
<code>createScript</code>	Builds the shell script that creates a VM and cleans up on failure
<code>planClone</code>	Computes the XML and file copies needed to clone a VM

The project does not yet include an automated test target. During development these functions were checked with throwaway harnesses (sample inputs, `xmllint` , and stand-in `virsh` and `qemu-img` scripts). Turning them into XCTest cases is a good next step.

UI structure

- `ContentView` is a `NavigationSplitView` . The sidebar lists sections, and the detail area switches on the selected `Section` .
- A banner at the top of the detail area shows action results.
- `ErrorsPanel` decides which errors are shown on which tab.
- Reusable pieces: `Card` , `UsageBar` , `Pill` , `Spark` .
- Sheets handle multi-field flows: share editor, VM create, VM edit, VM clone, VM details, container logs.
- `MenuBarExtra` provides the menu bar item. `Settings` provides the preferences window.

Security design

- Secrets (API key, SSH password) are stored in the Keychain.
- The SSH password reaches `ssh` only through an environment variable read by a helper script, so it is not written to disk or shown in process arguments.
- Destructive operations require explicit confirmation in the UI.
- The app connects only to the address you enter.

Adding a feature

1. Add the data model and a query or script. Prefer a separate, isolated query so a failure does not break other panels.
2. Add `@Published` state to `Store` and fill it in `refresh()` (or load it on demand from an action).
3. If it can fail, give it an error key, and decide in `ErrorsPanel` and `relevant(_:)` where it should show.
4. Add the view, and a sidebar `Section` if it needs its own tab.
5. For anything destructive, add a confirmation dialog.
6. Document the new queries or commands in [Data sources](#).

© 2026 Ray Munro. Licensed under the GNU General Public License v3.0 or later.

Building and releasing

Requirements

- A Mac running macOS 14 or later
- Xcode (the full app, for the Swift toolchain and SDK) or the matching command line tools
- Optional: an Apple Development certificate, for stable code signing

Build the app

```
./build_app.sh
```

The script:

1. Builds a release binary with `swift build -c release`.
2. Creates `Unraid Watcher.app` with the executable, the icon, and an `Info.plist`.
3. Signs it with the first Apple Development certificate it finds, or ad-hoc if there is none.

Open it with:

```
open "Unraid Watcher.app"
```

For a quick debug build while developing, use `swift build` and run the executable from `.build`. Features that depend on being an app bundle (notifications, the app icon) only work from the `.app`.

Why signing matters

macOS ties Keychain access to the app's code signature. An ad-hoc signature changes on every build, so macOS asks for your login keychain password again after each rebuild. A stable Apple Development signature avoids that. To see your available identities:

```
security find-identity -v -p codesigning
```

Build the DMG installer

```
./make_dmg.sh
```

This builds the app and the documentation PDF, then produces `dist/Unraid-Watcher-<version>.dmg` containing the app, an Applications shortcut, `Documentation.pdf`, `LICENSE.txt`, and a styled background with an arrow. Steps performed:

1. Build and sign the app, and build the documentation PDF.
2. Draw the background (`tools/make_dmg_bg.swift`).
3. Create a writable disk image and arrange the window using Finder scripting.
4. Compress it to a read-only image and sign it.

The first time, macOS may ask permission for your terminal to control Finder. Allow it for the styled window. If you decline, the DMG is still created and installs normally, just without the custom layout.

Build the documentation PDF

```
python3 tools/make_docs_pdf.py
```

This combines the README, every guide in `docs/` , and the changelog into `docs/Unraid-Watcher-Documentation.pdf` , with a cover page showing the app icon, a contents page, and page numbers. It needs only Python 3 and Google Chrome (used headless to print the PDF). Rebuild it whenever the documentation changes, then commit the new PDF.

`make_dmg.sh` runs this step automatically and puts the PDF in the installer as `Documentation.pdf` , next to a copy of the license as `LICENSE.txt` .

Regenerate the icon

```
swift tools/make_icon.swift
```

This redraws the icon in code and rewrites `Resources/AppIcon.icns` . Edit the drawing code in `tools/make_icon.swift` to change the design.

Changing the version

The version number appears in four places. Update all of them:

File	What to change
<code>build_app.sh</code>	<code>CFBundleShortVersionString</code> in the generated <code>Info.plist</code>
<code>make_dmg.sh</code>	<code>VER=</code> at the top (also names the DMG file)
<code>Sources/UnraidWatcher/App.swift</code>	<code>.applicationVersion</code> in the About panel
<code>Sources/UnraidWatcher/Views.swift</code>	The version text at the bottom of Settings

Also add an entry to [CHANGELOG.md](#).

Distributing to other Macs

A build signed with an Apple Development certificate runs on your own Mac. Other Macs show an unidentified developer warning, and the recipient must right-click the app and choose Open the first time.

To remove the warning for everyone you need a paid Apple Developer Program membership and these steps:

1. Sign with a **Developer ID Application** certificate and the hardened runtime:

```
codesign --force --options runtime --sign "Developer ID Application: Your Name (TEAMID)" "Unraid-Watcher.app"
```

2. Submit the DMG for notarization:

```
xcrun notarytool submit dist/Unraid-Watcher-1.0.dmg --keychain-profile "your-profile" --wait
```

3. Staple the ticket:

```
xcrun stapler staple dist/Unraid-Watcher-1.0.dmg
```

Files that are not committed

`.gitignore` excludes `.build/`, `dist/`, and `*.app`. They are regenerated by the scripts above.

© 2026 Ray Munro. Licensed under the GNU General Public License v3.0 or later.

Changelog

All notable changes to Unraid Watcher are listed here.

1.0.0

First release.

- **Dashboard:** overview, system and hardware details, UPS, license and services
- **Storage:** array and disk status and temperatures, parity history, mounted filesystems, SMART health per disk with self-tests
- **Performance:** per-core CPU, memory breakdown, disk I/O, fans, CPU temperatures, network traffic, top processes, system log
- **Docker:** start, stop, restart, pause, resume, and container logs
- **VMs:** power actions, details, creation wizard, editing (basics, media, full XML), cloning, install media eject, and console access
- **Shares:** settings, per-disk usage, create, edit, and delete empty shares
- **Controls:** array start and stop, parity checks, disk spin up and down, mover, reboot and shutdown, SSH command console
- **Alerts:** macOS notifications for hot disks, hot CPU, SMART problems, and new Unraid alerts, with adjustable thresholds
- **Menu bar** status item with a quick summary
- **Security:** API key and SSH password stored in the Keychain, SSH by key or password, confirmation for destructive actions
- **Packaging:** app icon, About window, build scripts, and a DMG installer
- **License:** GNU General Public License v3.0 or later

© 2026 Ray Munro. Licensed under the GNU General Public License v3.0 or later.